

IA: MACHINE LEARNING FOUNDATIONS

Unsupervised Learning & Clustering

Adrián Sánchez Morales

AGENDA

1. Introduction

2. Dimensionality Reduction
   - 2.1. Principal Component Analysis
   - 2.2. Singular Value Decomposition
   - 2.3. t-SNE

3. Clustering
   - 3.1. k-Means
   - 3.2. Hierarchical Clustering
   - 3.3. DBSCAN



1. INTRODUCTION

- Most of this subject focuses on supervised learning.
- Today, we deep dive into unsupervised methods, a set of tools intended for the setting in which we have only a set of features but no response → no prediction.

Is there an informative way to visualize the data?

Can we discover subgroups among the variables or among the observations?

- Unsupervised learning is often much more challenging than supervised learning.



1. INTRODUCTION

- Most of this subject focuses on supervised learning.
- Today, we deep dive into unsupervised methods, a set of tools intended for the setting in which we have only a set of features but no response → no prediction.

Is there an informative way to visualize the data?

Can we discover subgroups among the variables or among the observations?

- Unsupervised learning is often much more challenging than supervised learning.

1. There is no way to check our work

2. It is generally more subjective

1. INTRODUCTION

- Unsupervised learning techniques has grown importance over the last years:
    1. Cancer research
    2. Fake news
    3. Spam filter
    4. Marketing and sales
    5. Fraud detection

- In unsupervised learning the learning algorithm focuses solely on detecting structure in unlabeled input data. One generally differentiates between:
    1. **Clustering**, where the goal is to find homogeneous subgroups within the data; the grouping is based on distance between observations.
    2. **Dimensionality reduction**, where the goal is to identify patterns in the features of the data. Dimensionality reduction is often used to facilitate visualization of the data, as well as a pre-processing method before supervised learning.

- In real world, there is generally much more supervised problems than unsupervised problems.

AGENDA

1. Introduction

2. Dimensionality Reduction
   - 2.1. Principal Component Analysis
   - 2.2. Singular Value Decomposition
   - 2.3. t-SNE

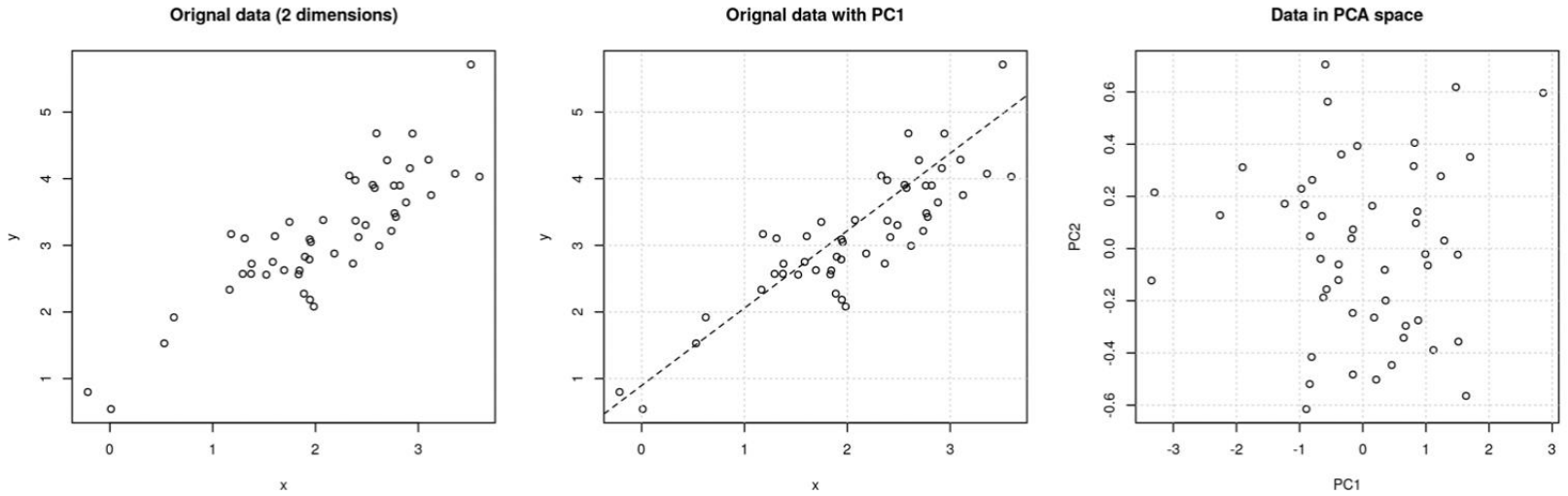
3. Clustering
   - 3.1. k-Means
   - 3.2. Hierarchical Clustering
   - 3.3. DBSCAN



2. DIMENSIONALITY REDUCTION

- **Dimensionality reduction** techniques are widely used and versatile techniques that can be used to:
  1. Find structure in features
  2. Pre-processing for other ML algorithms
  3. Aid in visualization

- The basic principle of dimensionality reduction techniques is to transform the data into a new space that summaries properties of the whole data set along a reduced number of dimensions. These are then ideal candidates used to visualize the data along these reduced number of informative dimensions.



2. DIMENSIONALITY REDUCTION

2.1. PRINCIPAL COMPONENT ANALYSIS

What is data variation? Why is it important?



2. DIMENSIONALITY REDUCTION

2.1. PRINCIPAL COMPONENT ANALYSIS

What is data variation? Why is it important?

- Variation is a way to show how data is dispersed or spread out. It shows how far apart data points are from one another.
- This is important because the amount of variability determines how well you can generalize results from the sample to your population.
- The higher variance a machine learning model can capture the better.

2. DIMENSIONALITY REDUCTION

2.1. PRINCIPAL COMPONENT ANALYSIS

- We want to find a low-dimensional representation of the data that captures as much of the information as possible.
- PCA seeks a small number of dimensions that are as interesting as possible.
- Theoretically*:
  - The firs principal component Z_1 defines a direction in feature space along which **the data vary the most**
  - The second principal component is a linear combination of predictors that has **maximal variance** out of all linear combinations that are uncorrelated with Z_1
- Once we have all the PCs we can plot them to against each other in order to produce low-dimensional views of the data.

* NOTE: find mathematical details about calculating principal components in the recommended books.

2. DIMENSIONALITY REDUCTION

2.1. PRINCIPAL COMPONENT ANALYSIS - STEPS

1. **Standardization**: The data are normalized so that each variable has a mean of zero and a variance of one.

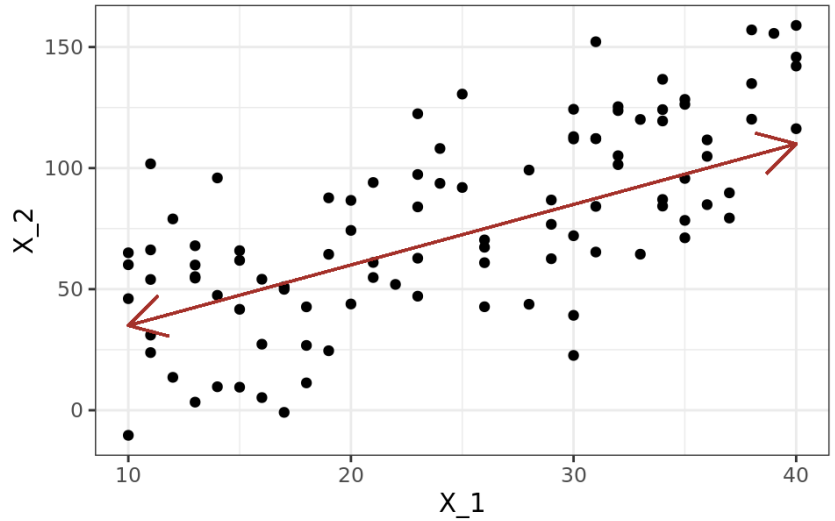
2. **Calculation of the covariance matrix**: The variables' variation in relation to each other is analyzed.

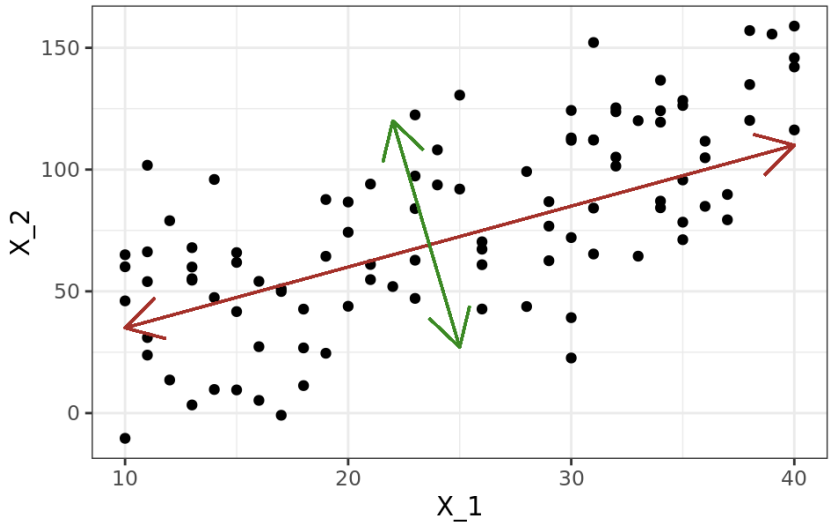
3. **Obtaining the eigenvectors and eigenvalues**: These define the directions (principal components) and the magnitude of the explained variance.

4. **Ordering the Principal Components**

5. **Data projection**: The original data are transformed into the new space defined by the principal components.

6. **Choosing the Optimal Number of Components**





2. DIMENSIONALITY REDUCTION

2.1. PRINCIPAL COMPONENT ANALYSIS – MAIN STEPS

2. Calculation of the covariance matrix:

The covariance matrix describes how variables vary together.

For a dataset with n variables, the result is an $n \times n$ matrix.

Each element (i, j) represents the **covariance** between variable i and variable j.

3. Eigenvectors and Eigenvalues Computation

This step identifies the **principal directions** (eigenvectors) and how much **variance** (eigenvalues) they explain. We solve the **eigenvalue problem**:

$$\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$$

Where:
$\mathbf{C}$ is the covariance matrix,
λ is the eigenvalue (explained variance),
$\mathbf{v}$ is the eigenvector (component direction).

Eigenvectors define the **principal components**.

Eigenvalues indicate the **relative importance** of each component.

5. Projecting the Data into the New Space

The original dataset is transformed into the new space defined by the principal components:

$$Z = X \cdot W$$

Where:
X is the standardized data matrix,
W is the matrix of selected eigenvectors,
Z is the projected data matrix (reduced dimensionality).

2. DIMENSIONALITY REDUCTION

2.1. PRINCIPAL COMPONENT ANALYSIS – MAIN STEPS

- **Solving the Eigenvalue Problem in PCA**

Given a **covariance matrix** $\mathbf{C}$ of dimension $n \times n$, the goal is to find the **eigenvalues** λ and **eigenvectors** $\mathbf{v}$ that satisfy:

$$\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$$

This is known as an **eigenvalue problem**, where: λ represents the **variance explained** by the component.

$\mathbf{v}$ is the **direction** (a unit vector) in the space of the original variables.

- **Steps to Solve It:**

1. Form the Covariance Matrix

From the standardized data matrix X, compute:

$$\mathbf{C} = \frac{1}{n-1} X^T X$$

2. .Solve the Characteristic Equation

Find the values λ that satisfy:

$$\mathbf{det}(\mathbf{C} - \lambda\mathbf{I}) = 0$$

This yields a **characteristic polynomial** of degree n, whose roots are the **eigenvalues**.

3. Compute the Eigenvectors

For each eigenvalue λ_i ,solve:

$$(\mathbf{C} - \lambda_i\mathbf{I})\mathbf{v}_i = 0$$

This is typically done using **numerical methods**, such as **Eigen decomposition** (spectral decomposition) or **Singular Value Decomposition (SVD)**, which is more numerically stable and widely used in modern PCA implementations.

2. DIMENSIONALITY REDUCTION

2.1. PRINCIPAL COMPONENT ANALYSIS

Scaling Variables

Variables should be center to have zero mean

Results are sensitive to have individually scaled variables

Uniqueness

Each principal component loading vector is unique

Number of PCs

It will depend on the application and the dataset

Data visualization?

Improve regression?

2. DIMENSIONALITY REDUCTION

2.2. SINGULAR VALUE DECOMPOSITION

- The goal of singular value decomposition (SVD) is to decompose a matrix into simpler components: orthogonal and diagonal matrices.
- Specifically, SVD consists of transforming a matrix into three simpler transformations:

$$\mathbf{A}_{nxp} = \mathbf{U}_{nxn}\, \mathbf{S}_{nxp}\, \mathbf{V}^{\mathbf{T}}_{pxp}$$

$\boldsymbol{U}$ are the left singular vectors (orthogonal matrix)

$\boldsymbol{S}$ has singular values and is diagonal

$\boldsymbol{V^T}$ has rows that are the right singular vectors (orthogonal)

2. DIMENSIONALITY REDUCTION

2.2. SINGULAR VALUE DECOMPOSITION

How SVD turns data into principal components

1 Center

$X = \text{data} - \text{mean}$

Subtract each variable's mean.

Rows are observations. Columns are variables.

2 Decompose

$X = U \Sigma V^T$

V defines orthogonal axes.

Σ ranks singular values.

UΣ gives coordinates on those axes.

3 Select

$\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$

Keep the first k axes with the largest singular values.

Variance share of axis i:

$\sigma_i^2 / \sum_j \sigma_j^2$

4 Project

$Z = XV_k = U_k\Sigma_k$

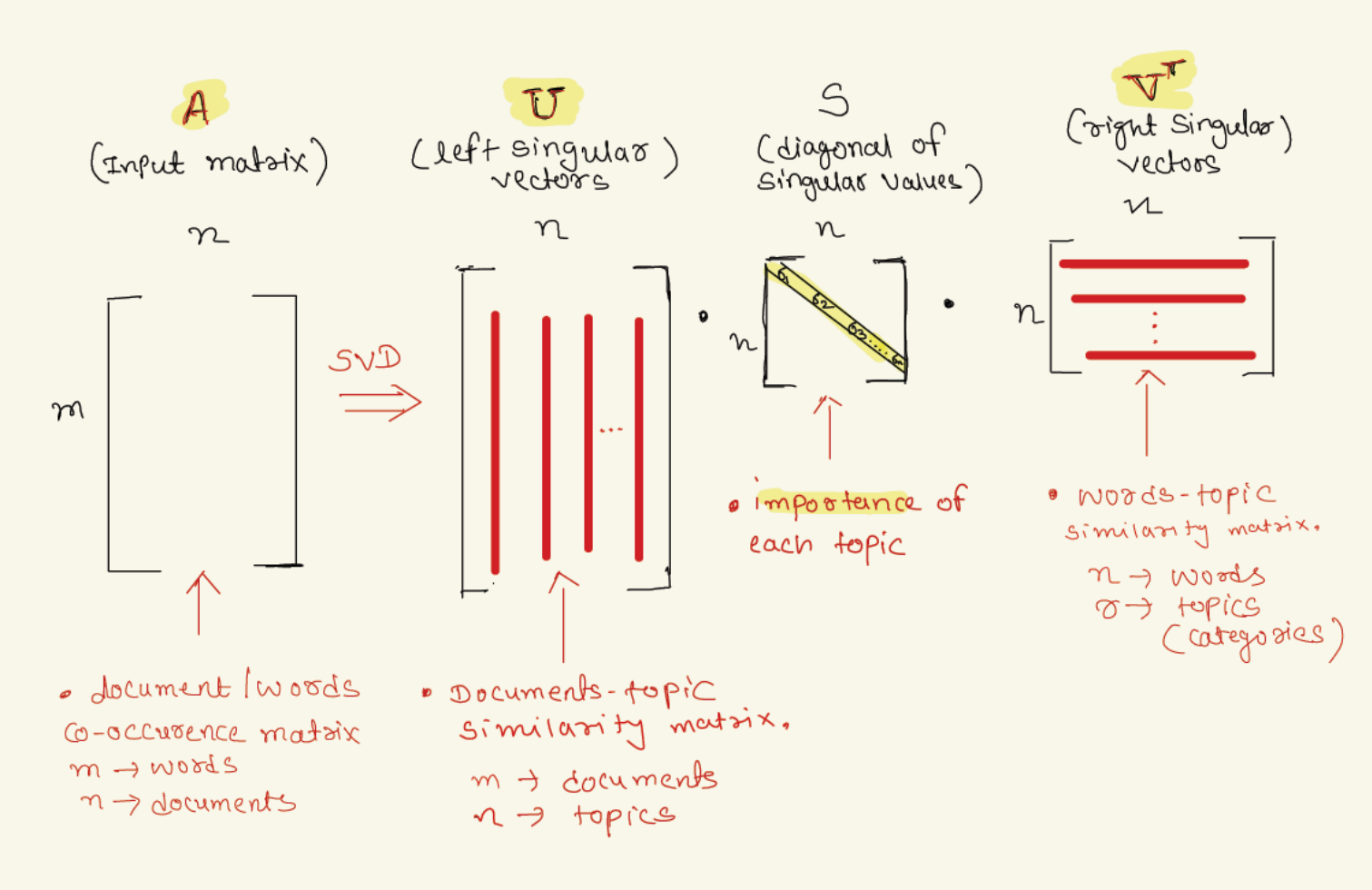
Express each observation using k coordinates.

These are the retained PCA scores.

SVD factorizes the data. Keeping only k components reduces dimensionality.

2. DIMENSIONALITY REDUCTION

2.2. SINGULAR VALUE DECOMPOSITION



2. DIMENSIONALITY REDUCTION

2.3. t-SNE

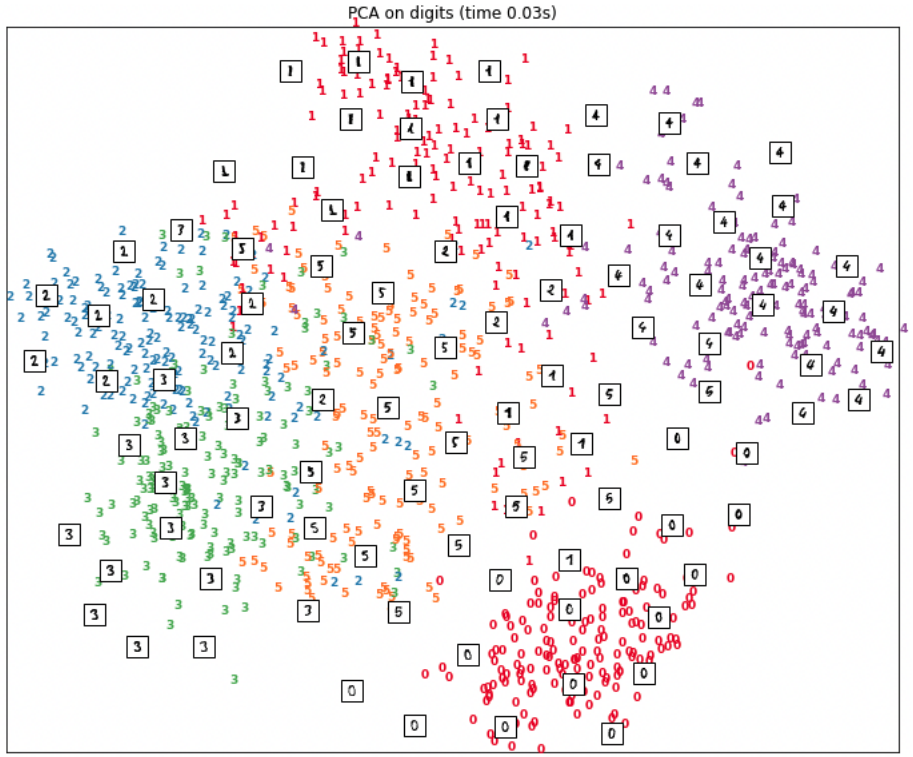
- t-Distributed Stochastic Neighbour Embedding (t-SNE) is a non-linear dimensionality reduction technique, i.e., different regions of the data space will be subjected to different transformations.
- The primary use of t-SNE is to visualize and explore the higher dimensional data.
- t-SNE will compress small distances, thus bringing close neighbors together, and will ignore large distances.
- It is particularly well suited for very high dimensional data.
- This is one of the best dimensionality reduction techniques especially for visualization of data

2. DIMENSIONALITY REDUCTION

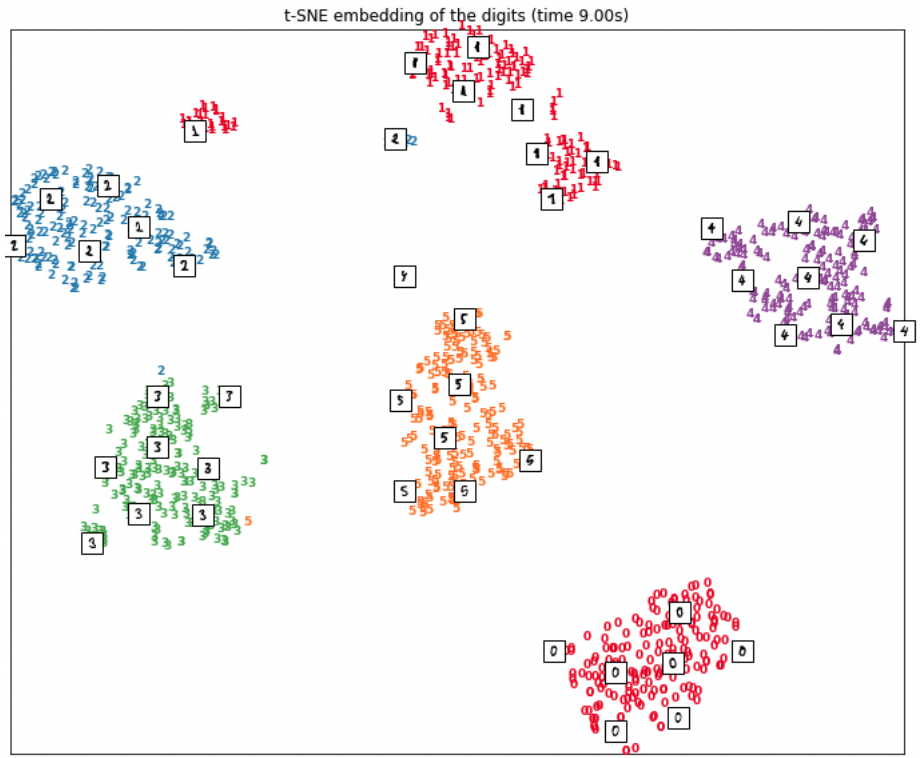
2.3. t-SNE

MNIST Example

PCA



t-SNE



2. DIMENSIONALITY REDUCTION

2.4. AUTOENCODERS

Learning a compact representation through reconstruction

An autoencoder learns to reproduce its input through a smaller latent space, without class labels.

ENCODER

$$z = f(x)$$

Compresses p input features into k learned features.

LATENT CODE

$$z \in \mathbb{R}^k, \; k < p$$

The bottleneck stores the reduced representation.

DECODER

$$\hat{x} = g(z)$$

Reconstructs the original p features from the code.

Training: adjust both networks to minimize reconstruction error, e.g. mean squared error.

After training, use the encoder alone to transform new observations into k features.

2. DIMENSIONALITY REDUCTION

2.4. AUTOENCODERS

Autoencoders and PCA: choosing and validating the representation

PCA

Linear projection onto orthogonal axes.

Components rank by explained variance.

A simple baseline with few tuning choices.

NONLINEAR AUTOENCODER

Learns nonlinear compression functions.

Latent features have no default variance ranking.

Requires architecture and training choices.

A practical workflow

1 Prepare

Split data first. Fit scaling and the autoencoder on training data only.

2 Tune

Choose k and model capacity using validation data. Control overfitting.

3 Evaluate

Compare with PCA at the same k on unseen data and the downstream task.

AGENDA

1. Introduction

2. Dimensionality Reduction
   - 2.1. Principal Component Analysis
   - 2.2. Singular Value Decomposition
   - 2.3. t-SNE

3. Clustering
   - 3.1. k-Means
   - 3.2. Hierarchical Clustering
   - 3.3. DBSCAN



3. CLUSTERING

- *Clustering* refers to a very broad set of techniques for finding subgroups, or clusters, in a data set.
    - The observations within each group must be similar to each other
    - Observations in different groups must be quite different from each other

- What does *similarity* mean?

- Clustering and PCA seek to simplify data, but their mechanisms are different:
    - PCA looks to find a low-dimensional representation keeping a good fraction of the variance
    - Clustering looks to find homogeneous subgroups among the observations

Recommendation Engines

Market and Customer Segmentation

Social Network Analysis

Search Result Clustering

Biological or Medical Analysis

3. CLUSTERING

3.1. K-MEANS

- K-means clustering is a simple and elegant approach for partitioning a data set into K distinct, non-overlapping clusters.

- The number of clusters K must be specified, and the algorithm will assign each observation to a cluster.

- The method creates sets such that:

$$C_1 \cup C_2 \cup \ldots \cup C_K = \{1, \ldots, n\} \qquad C_k \cap C_{k'} = \emptyset \text{ for all } k \neq k'$$

- The idea is having good clusters ⟶ the ones for which the within-cluster variation is as small as possible.

- Using Euclidean distance as measurement of variation:

$$\underset{C_1,\ldots,C_K}{\text{minimize}} \left\{ \sum_{k=1}^{K} W(C_k) \right\} \qquad W(C_k) = \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2$$

3. CLUSTERING

3.1. K-MEANS

- Now, we need an algorithm to solve these equations ⟶ There are K^n ways to partition n observations into K clusters

K-Means Clustering

1. Randomly assign a number, from 1 to K, to each of the observations. These serve as initial cluster assignments for the observations.

2. Iterate until the cluster assignments stop changing:

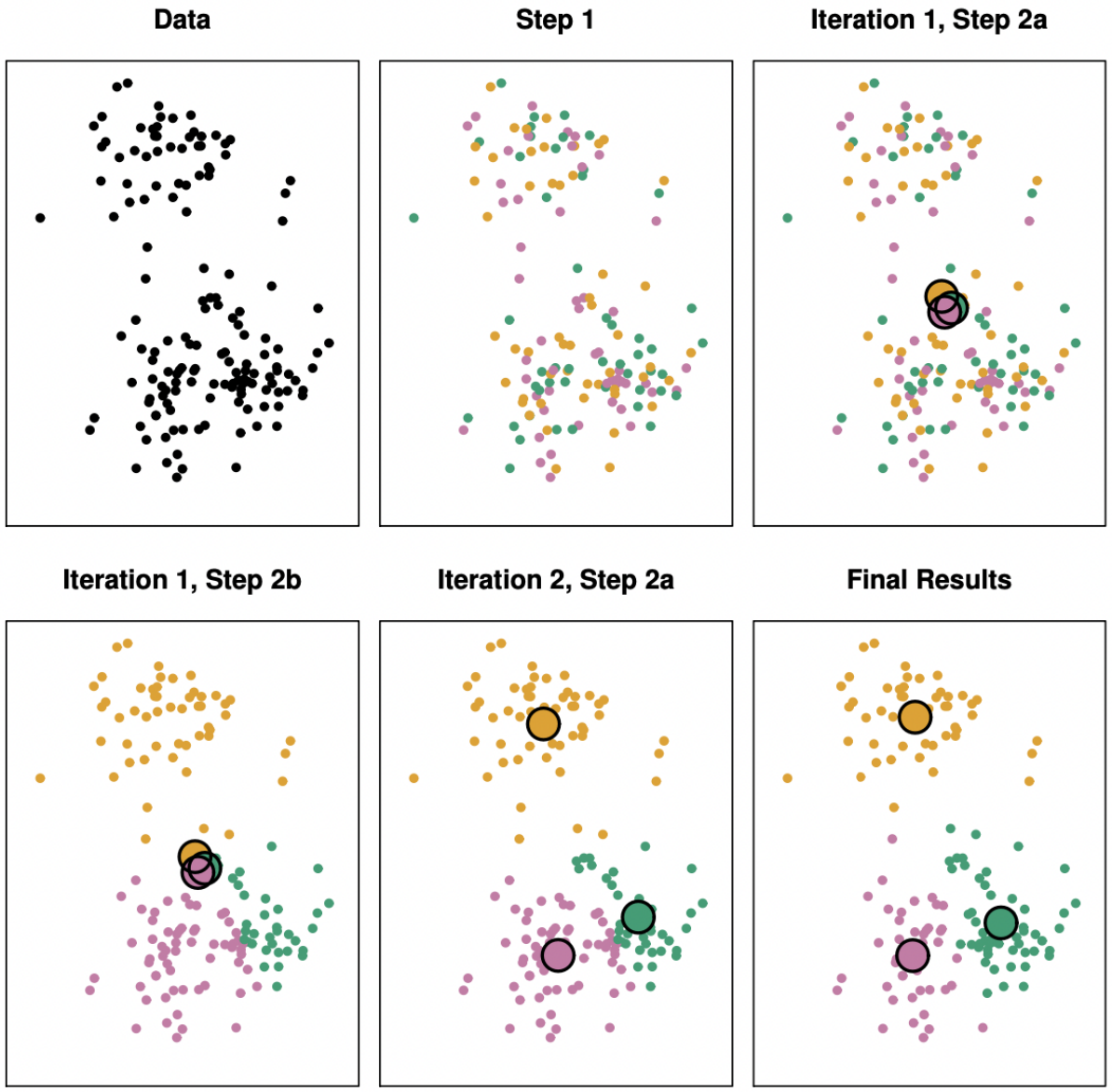
   (a) For each of the K clusters, compute the cluster *centroid*. The kth cluster centroid is the vector of the p feature means for the observations in the kth cluster.

   (b) Assign each observation to the cluster whose centroid is closest (where *closest* is defined using Euclidean distance).

- When the result no longer changes, a local optimum has been reached

3. CLUSTERING

3.1. K-MEANS



Example Video

3. CLUSTERING

3.1. K-MEANS

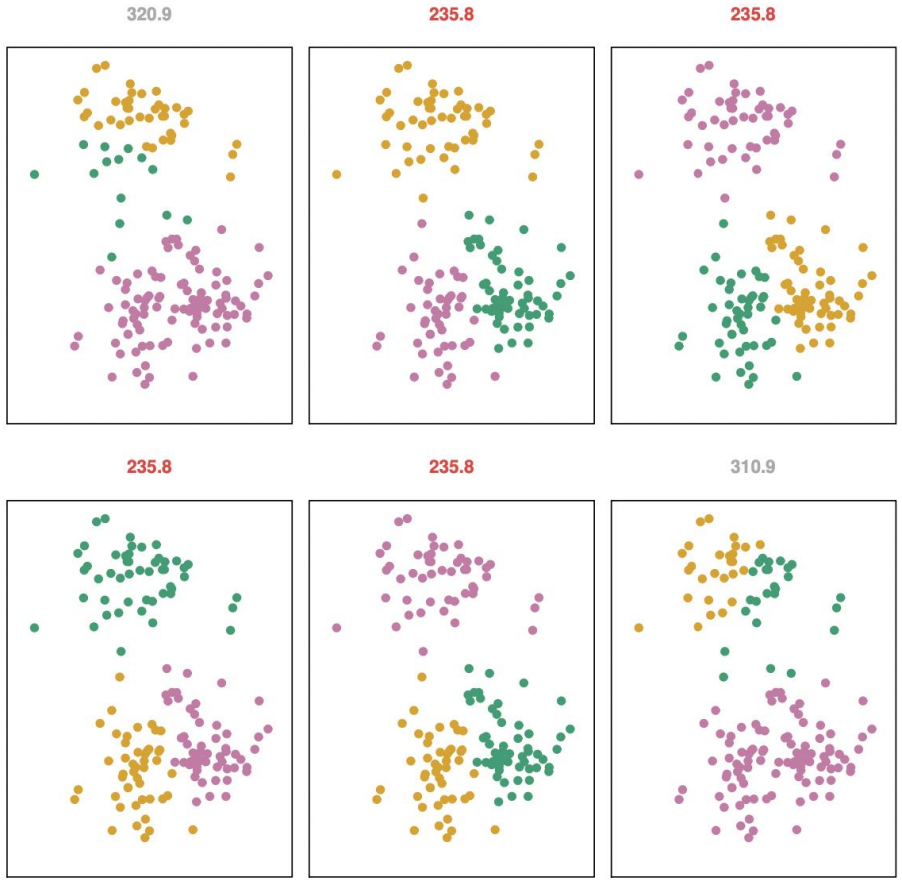
Is it necessary to run the algorithm multiple times?

3. CLUSTERING

3.1. K-MEANS

- Since the algorithm finds a local rather than a global optimum, the results will depend on the initial (random) cluster assignment of each observation.

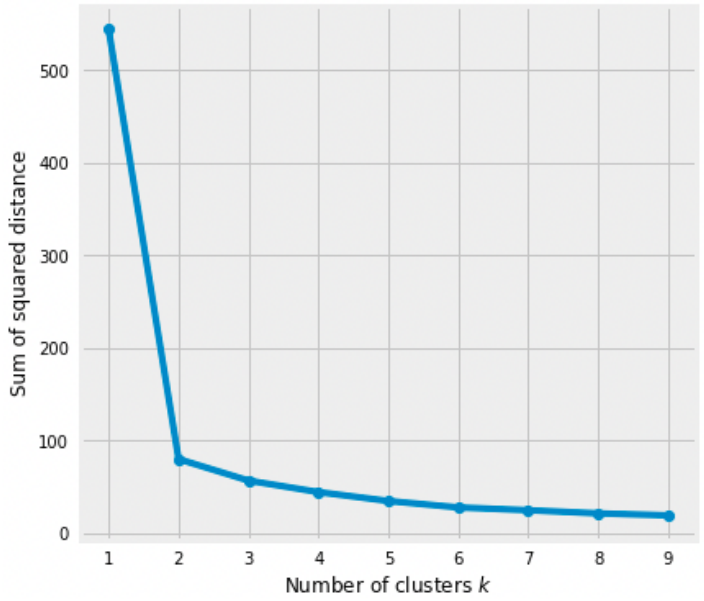
- We finally select the best solution, i.e., that for which the objective is smallest



3. CLUSTERING

3.1. K-MEANS

- An inappropriate choice of the number of clusters K may yield poor results.
- Elbow method gives us an idea on what a good K number of clusters would be based on the sum of squared distance (SSE) between data points and their assigned clusters' centroids.
- We pick K at the spot where SSE starts to flatten out and forming an elbow.



3. CLUSTERING

3.1. K-MEANS

Alternatives to the elbow method?

3. CLUSTERING

3.1. K-MEANS

Alternatives to the elbow method?

Beyond the Elbow Method

3. CLUSTERING

EXERCISE

Image Compression with K-Means:

$K = 2$



$K = 3$



$K = 10$



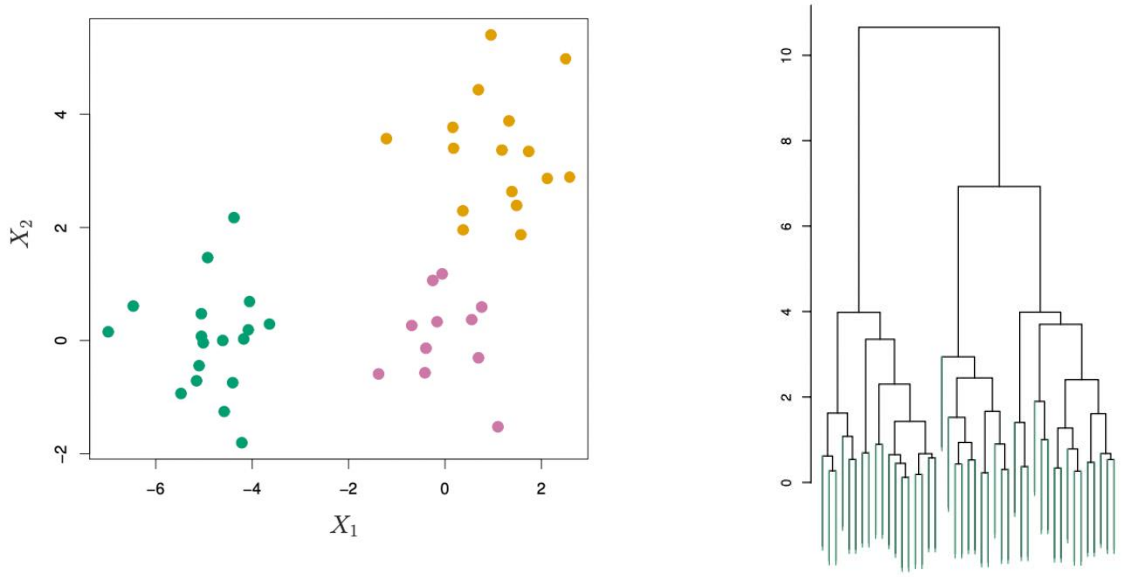
Original image



3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING

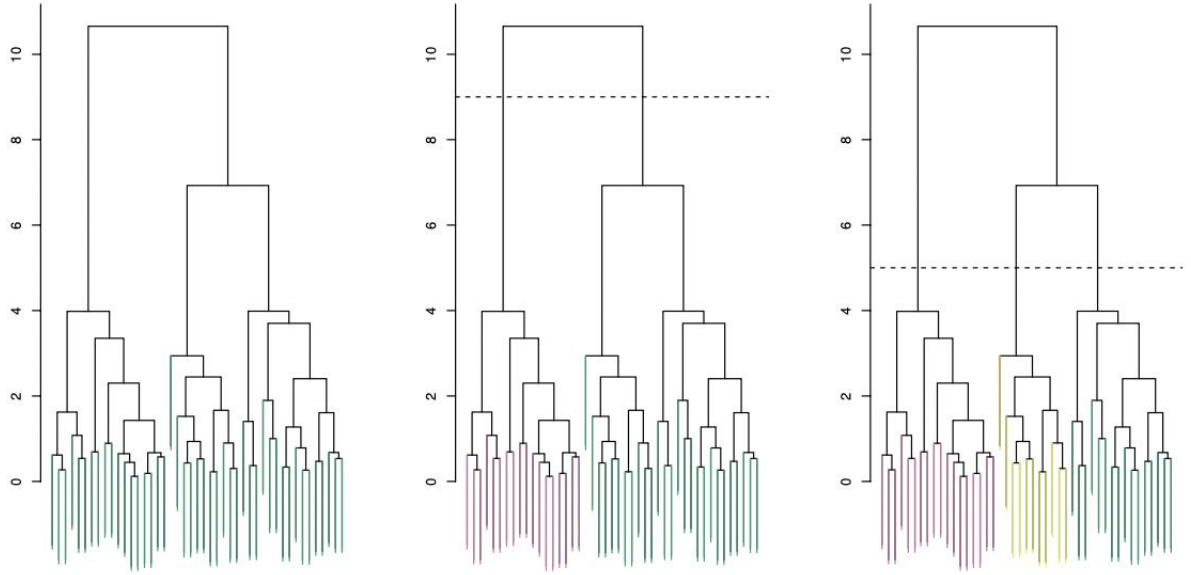
- The main drawback of K-means is that it requires us to pre-specify the number of clusters.
- *Hierarchical Clustering* solve this, and it gives results in an attractive tree-based representation of the observations called *dendrogram*.
- Here we focus on *bottom-up* or *agglomerative* clustering.



3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING

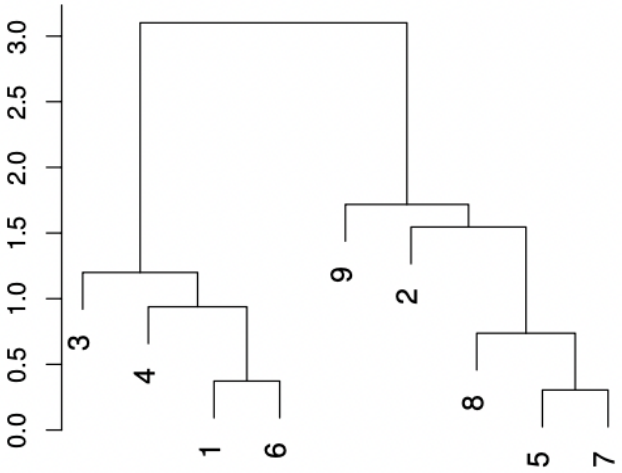
- The main drawback of K-means is that it requires us to pre-specify the number of clusters.
- *Hierarchical Clustering* solve this, and it gives results in an attractive tree-based representation of the observations called *dendrogram*.
- Here we focus on *bottom-up* or *agglomerative* clustering.

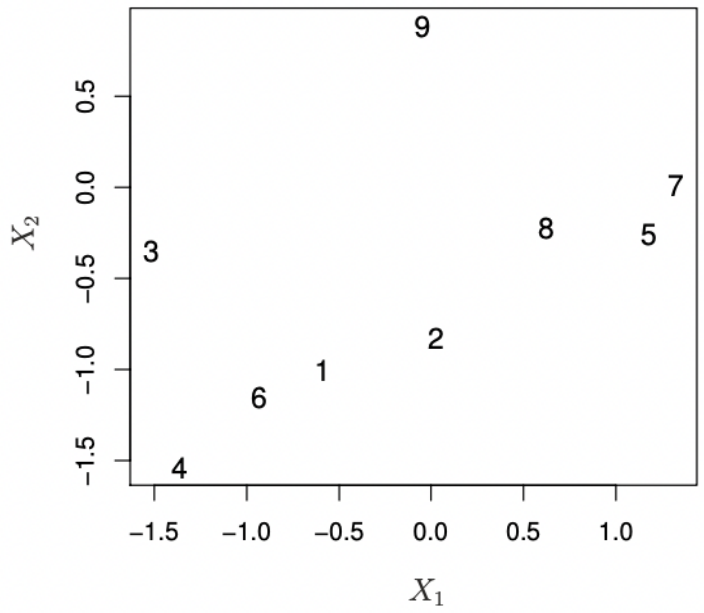


- Each leaf is a sample
- Branches represent similar samples
- The earlier fusions occur, the more similar
- For any pair of observations, we define similarity by their fusion point in the vertical axis
- Number of clusters are defined by horizontal cuts

3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING



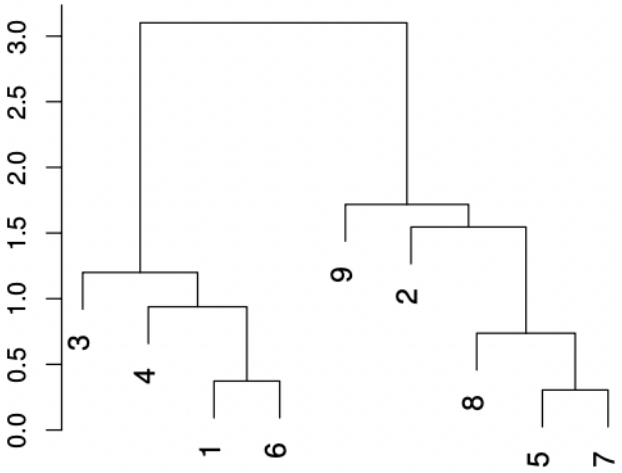


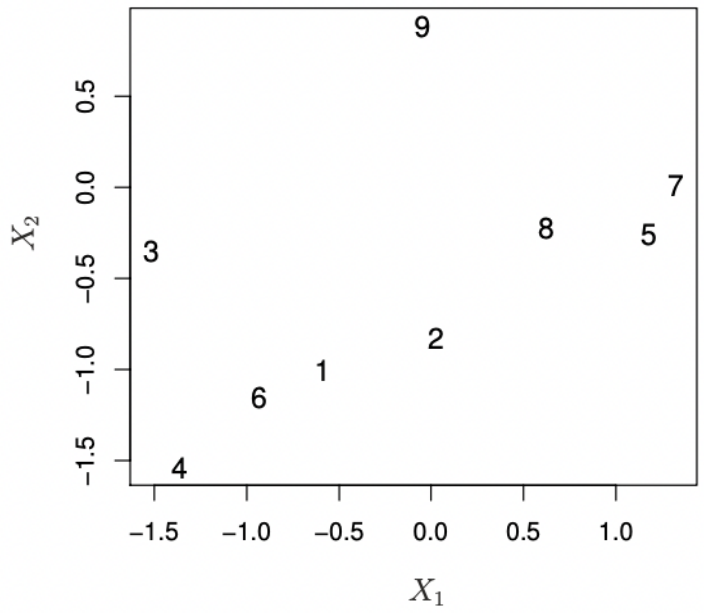
EXERCISE

- 5 and 7 similar?
- 1 and 6 similar?
- 9 and 2 similar?
- 9 more similar to 2 than to 8/5/7?

3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING





EXERCISE

- 5 and 7 similar? **YES**
- 1 and 6 similar? **YES**
- 9 and 2 similar? **NO**
- 9 more similar to 2 than to 8/5/7? **NO**

3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING

- The number of clusters is normally set by eye or domain knowledge.
- This method works well in scenarios with hierarchical structure data.

Hierarchical Clustering

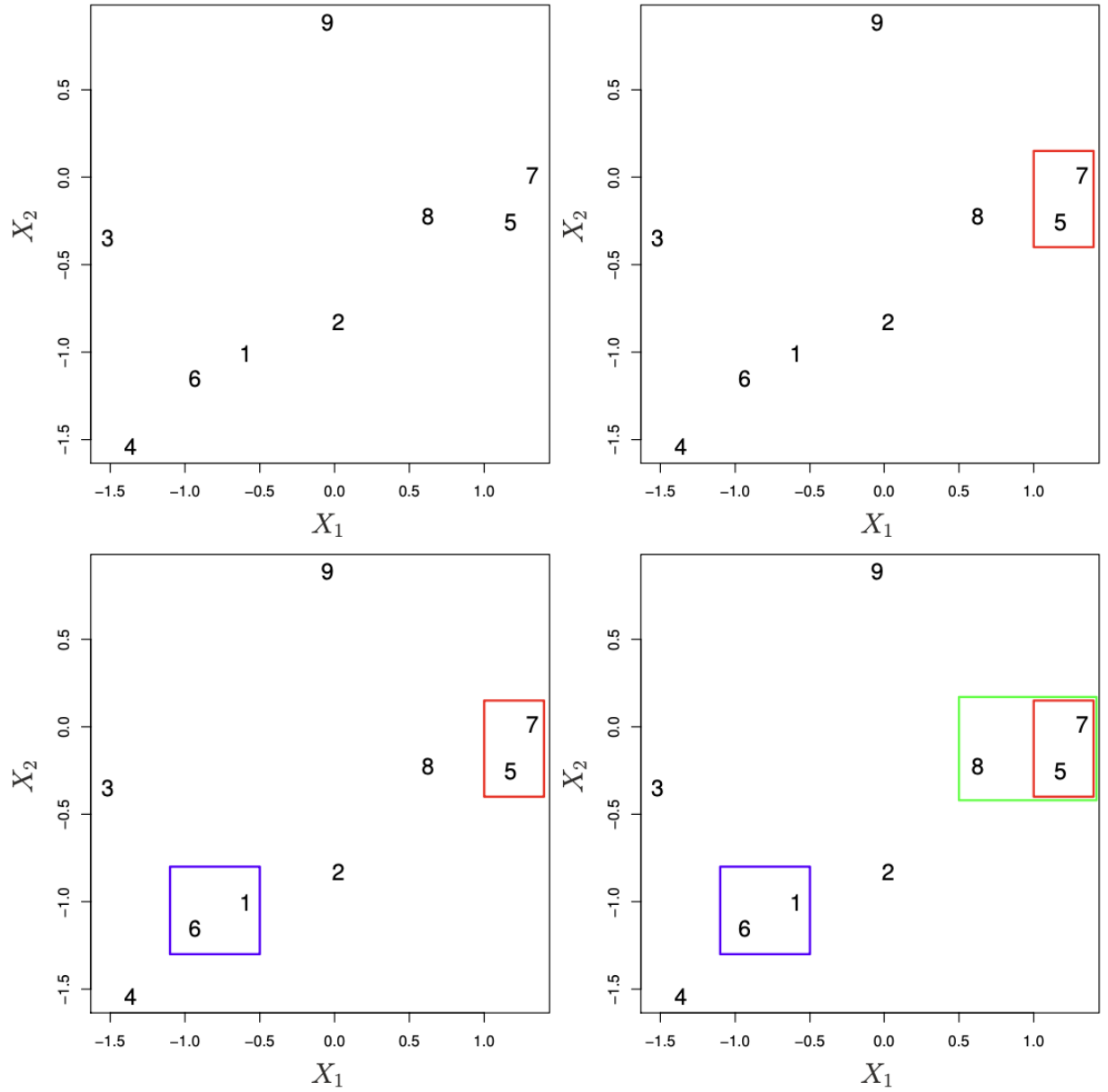
1. Begin with n observations and a measure (such as Euclidean distance) of all the $\binom{n}{2} = n(n-1)/2$ pairwise dissimilarities. Treat each observation as its own cluster.
2. For $i = n, n-1, \ldots, 2$:
   (a) Examine all pairwise inter-cluster dissimilarities among the i clusters and identify the pair of clusters that are least dissimilar (that is, most similar). Fuse these two clusters. The dissimilarity between these two clusters indicates the height in the dendrogram at which the fusion should be placed.
   (b) Compute the new pairwise inter-cluster dissimilarities among the $i-1$ remaining clusters.

3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING

Is that all?





3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING

- The concept of dissimilarity between a pair of observations needs to be extended to a pair of groups of observations ⟶ *Linkage*

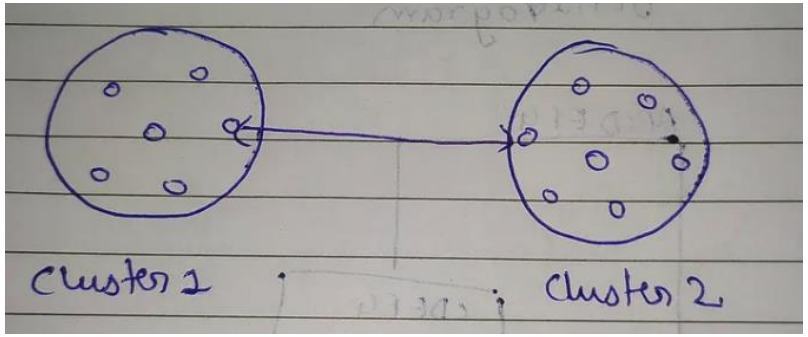
Linkage	*Description*
Complete	Maximal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B and record the largest of these dissimilarities.
Single	Minimal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B and record the smallest of these dissimilarities.
Average	Mean intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B and record the average of these dissimilarities.
Centroid	Dissimilarity between the centroid for cluster A (a mean vector of length p) and the centroid for cluster B.

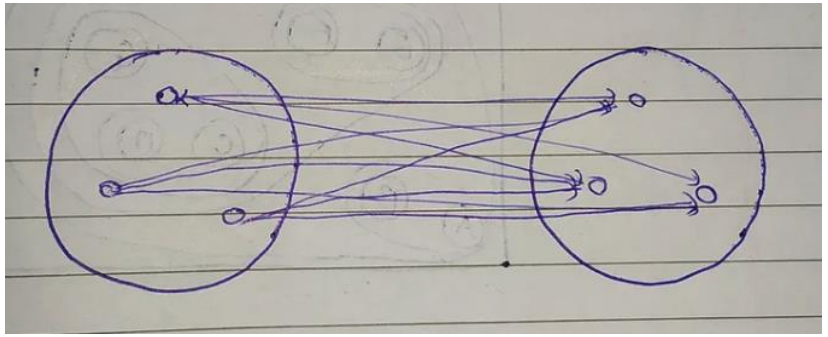
3. CLUSTERING

3.2. HIERARCHICAL CLUSTERING

- The concept of dissimilarity between a pair of observations needs to be extended to a pair of groups of observations ⟶ *Linkage*

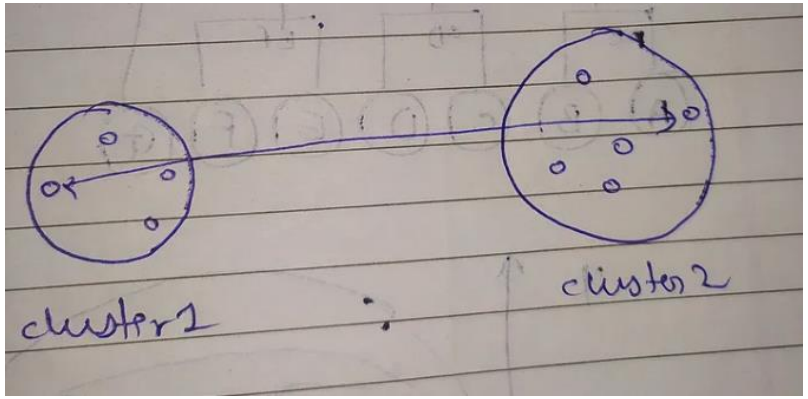
Single

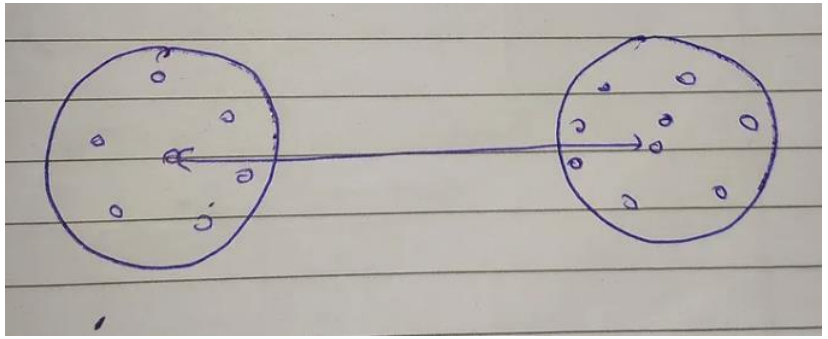




Average

Complete



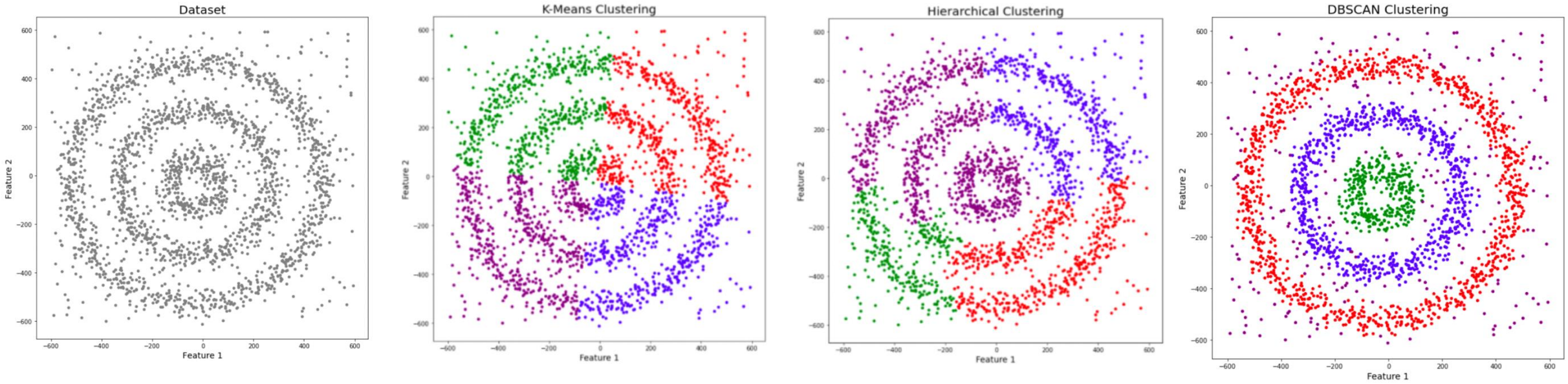


Centroid

3. CLUSTERING

3.3. DBSCAN

- K-means and Hierarchical Clustering are the most frequent methods, but they both fail in creating clusters of arbitrary shapes. They are not able to form clusters based on varying densities. That's why we need *DBSCAN clustering*.



3. CLUSTERING

3.3. DBSCAN

EXERCISE

Research about DBSCAN:

Topics:

- Definition
- Parameters
- Mathematical description
- Visual interpretation
- Applications
- Comparison with other models


THANK YOU